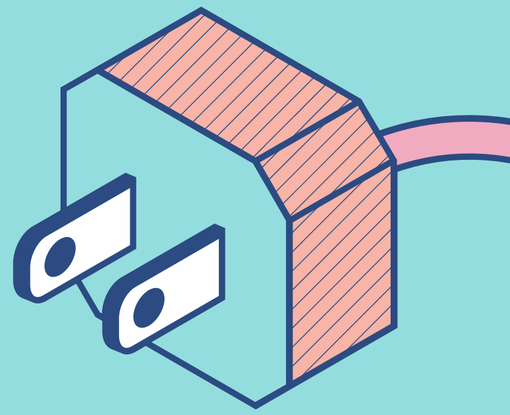
ATENEO DE ZAMBOANGA UNIVERSITY

Error Detection and Correction

ALEEKHAZER J. JAUSAN

CIT.016 | DATA COMMUNICATION

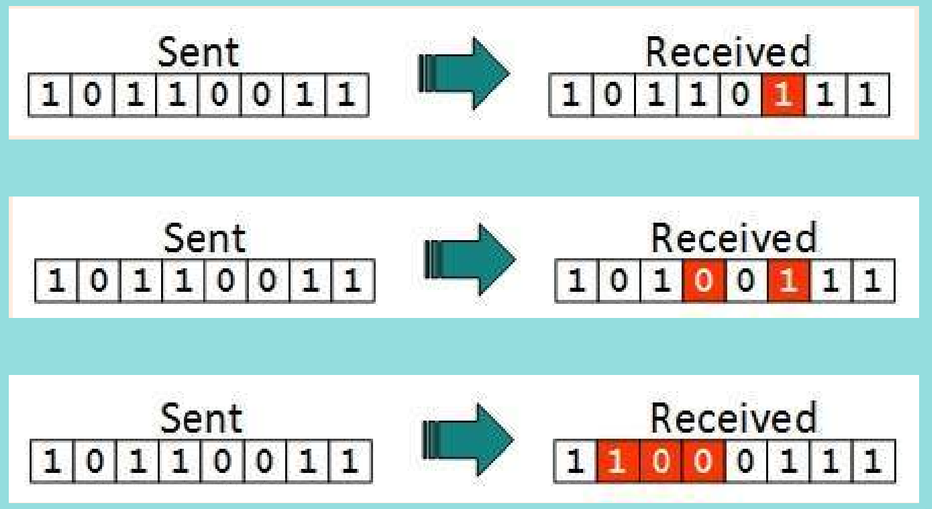


ERRORS

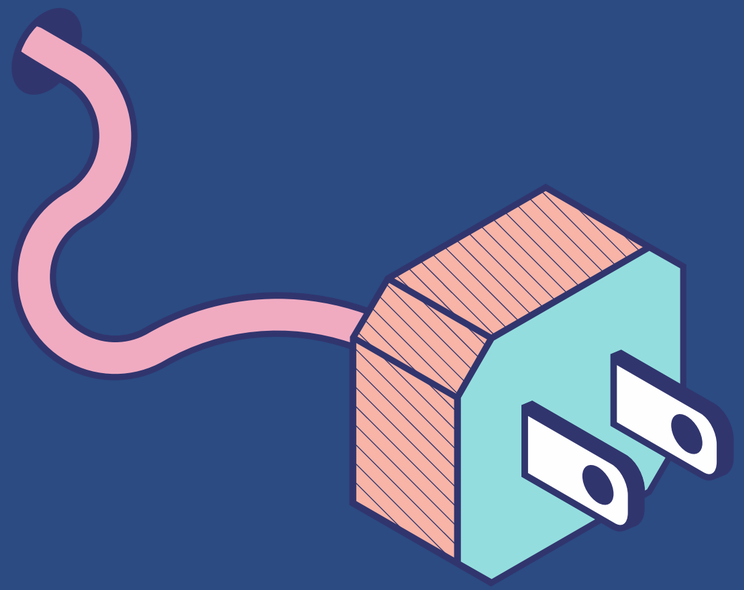
- Occurs when the information provided by the sender does not match that of the recipient
- Noise in digital signals can introduce errors in binary bits as they move from sender to receiver.

TYPES OF ERRORS

- **Single-Bit Error** - one bit of a transmitted data unit is changed during transmission
- **Multiple-Bit Error** - more than one bit in a data transmission is damaged
- **Burst Error** - several consecutive bits are incorrectly flipped in digital transmission



Error Detection Methods



Simple Parity Check

Simply adding an additional bit to a data transmission.

- If it contains an odd number of 1's, then add 1
- If it contains an even number of 1's, then add 0

Two-Dimensional Parity Check

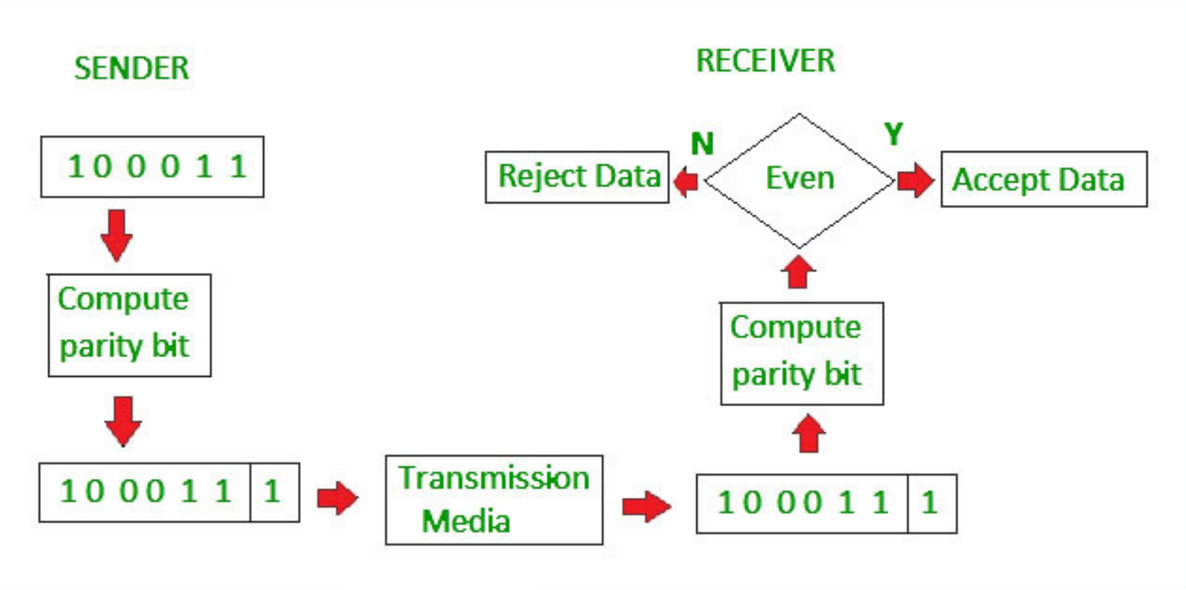
Parity check bits are calculated for all rows and columns, with both being sent along with the data.

Checksum

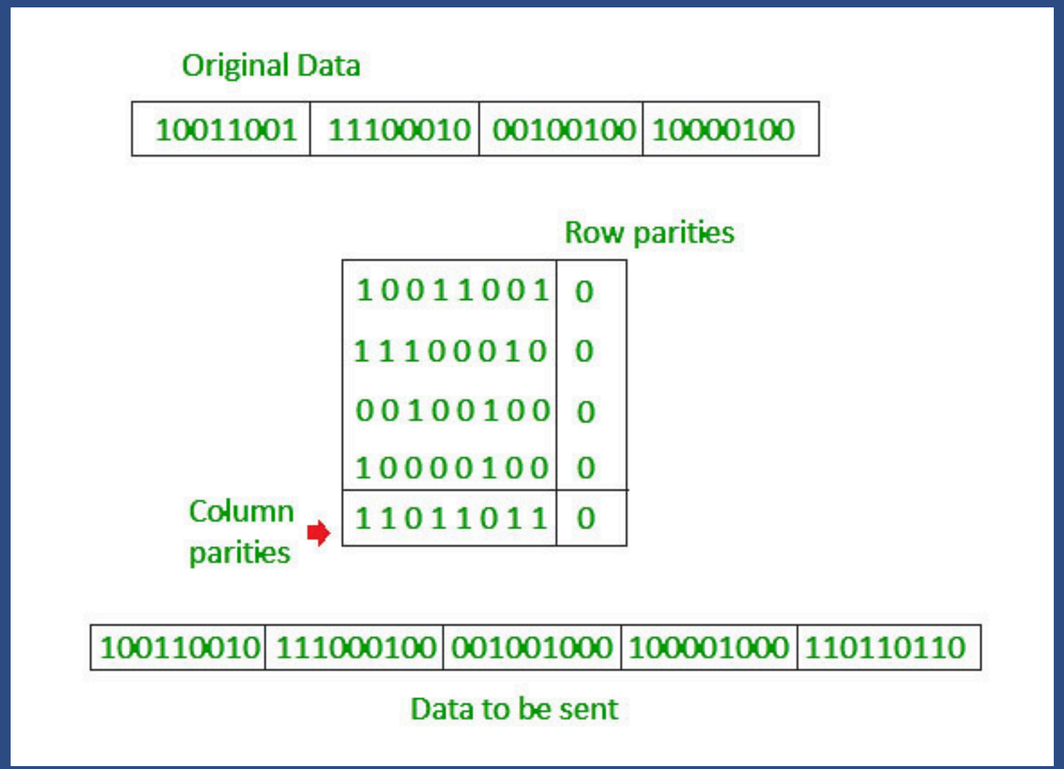
Involves dividing the data into equally sized segments, then calculating the sum of these segments using a 1's complement before sending this with the data to the receiver.

Error Detection Methods

Simple Parity Check

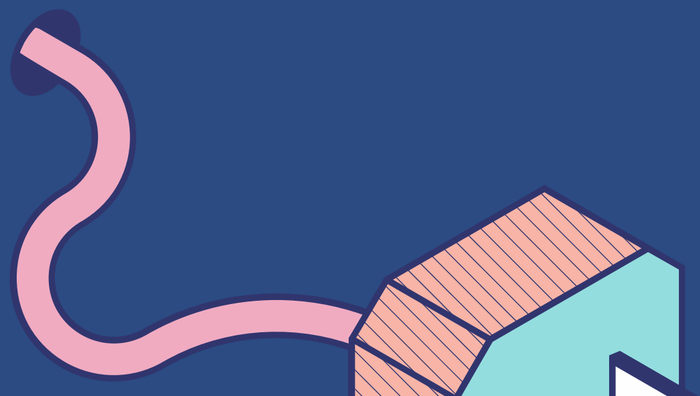


Two-Dimensional Parity Check



Error Detection Methods

Checksum



Original Data

10011001	11100010	00100100	10000100
1	2	3	4

k=4, m=8

Sender

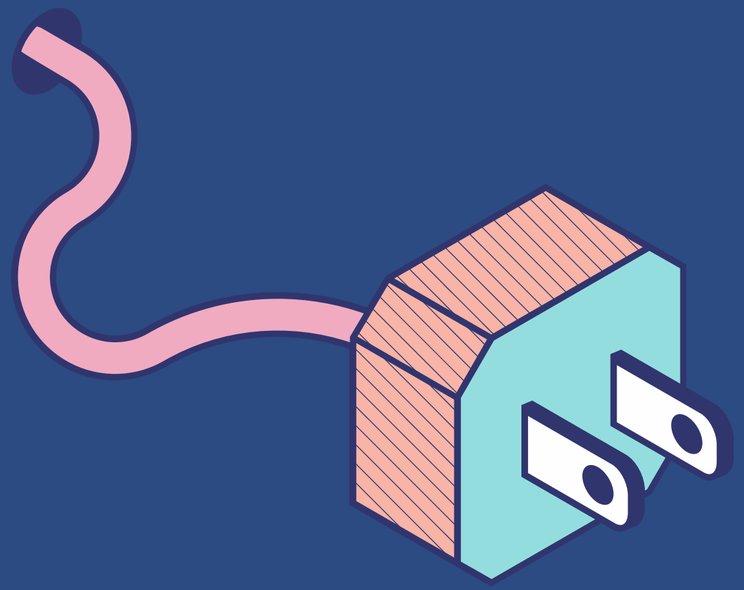
```
1   10011001
2   11100010
    ---------
    (1)01111011
          1
    ---------
    01111100
3   00100100
    ---------
    10100000
4   10000100
    ---------
    (1)00100100
          1
    ---------
Sum: 00100101
CheckSum: 11011010
```

Reciever

```
1   10011001
2   11100010
    ---------
    (1)01111011
          1
    ---------
    01111100
3   00100100
    ---------
    10100000
4   10000100
    ---------
    (1)00100100
          1
    ---------
    00100101
    11011010
    ---------
Sum: 11111111
Complement: 00000000
```

Conclusion: Accept Data

Error Detection Methods

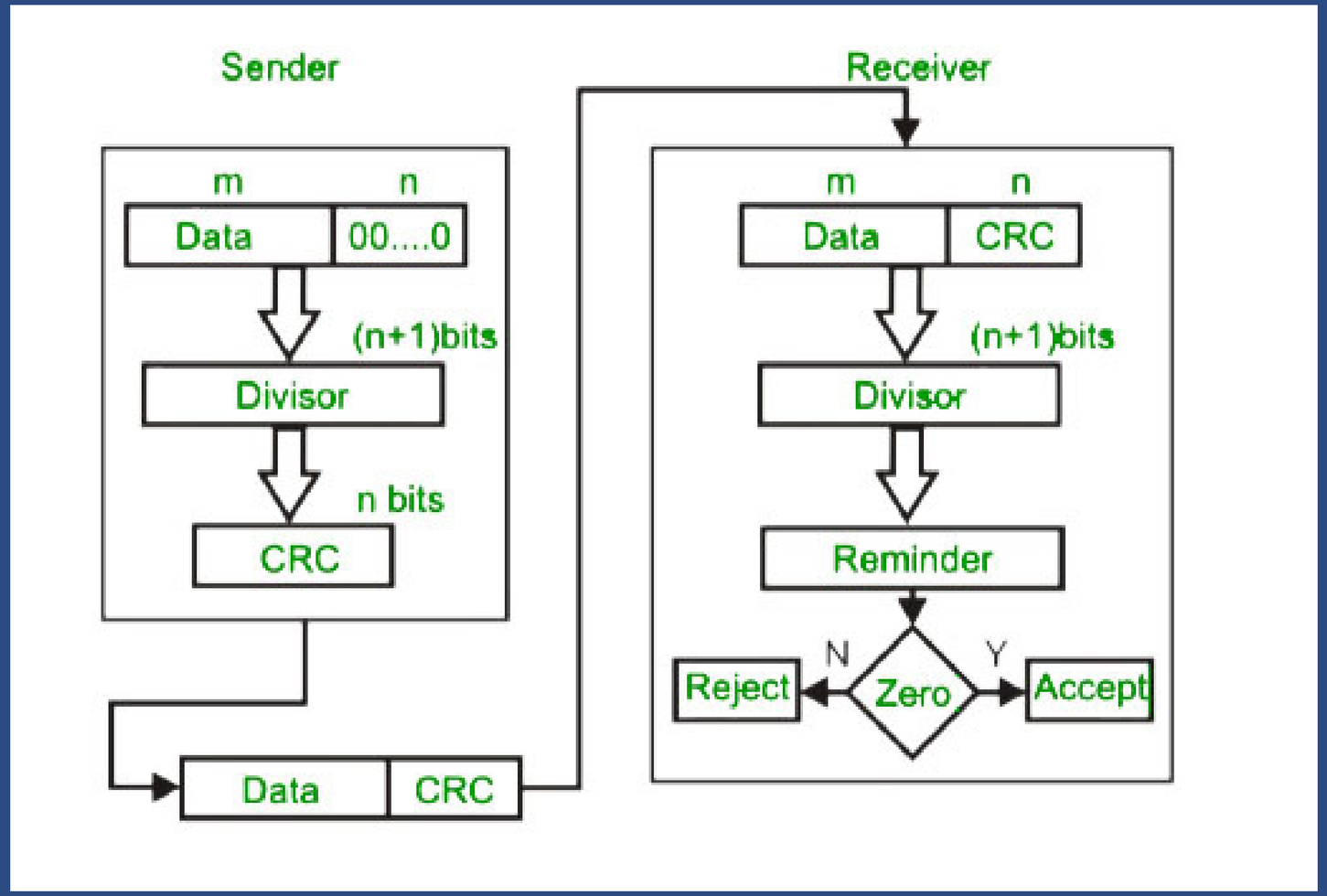


Cyclic Redundancy Check (CRC)

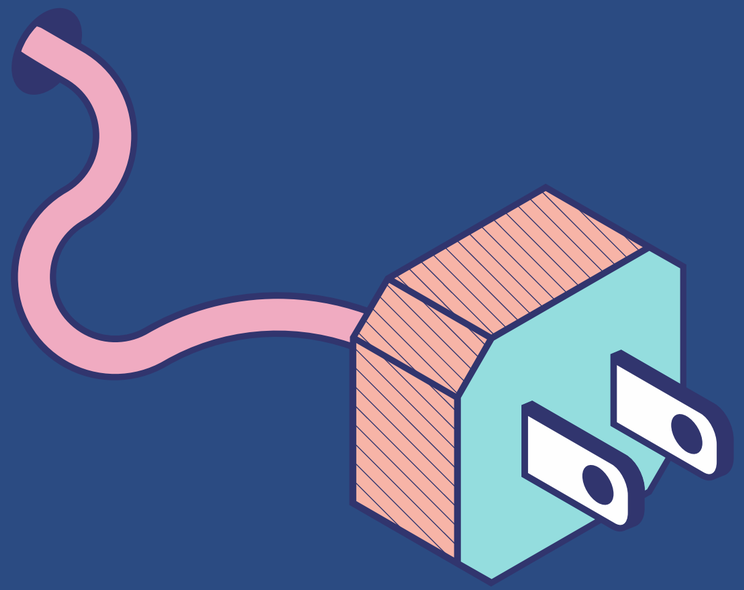
- A process based on binary division, unlike the checksum method
- Cyclic redundancy check bits are appended to the end of the data unit with the intent of making the resulting data is divisible by a second predetermined binary number
- This data unit is then sent and divided with the same number
- No remainder: data unit is correct and accepted
- Remainder: there has been damage done to the data unit during transmission, rejecting it

Error Detection Methods

Cyclic Redundancy Check (CRC)



Error Correction Methods



Backward Error Correction

When the receiver identifies an error in the data received, it requests that the sender retransmit the data unit

Forward Error Correction

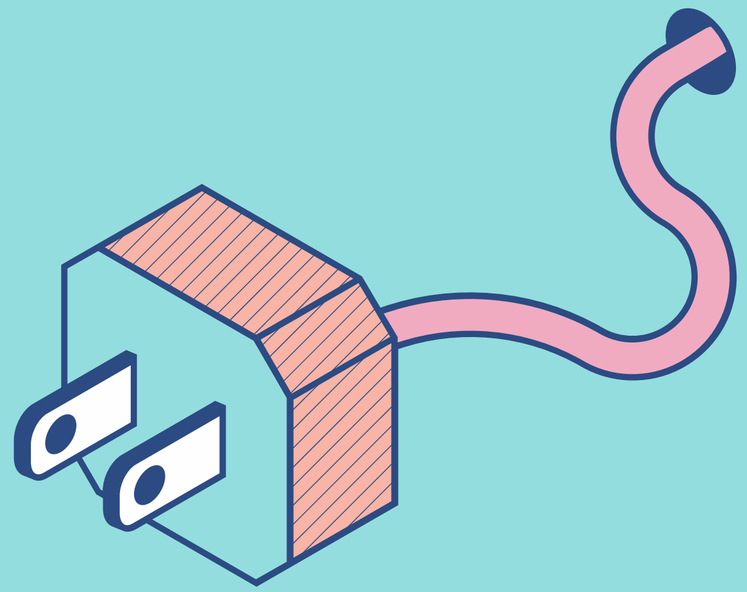
The receiver uses error-correcting code to help it auto-recover and fix certain types of errors when it finds errors in the data it has received

Hamming Code

Hamming code is an error correction system that can detect and correct errors when data is stored or transmitted. It requires adding additional parity bits with the data. It's named after its inventor, Richard W. Hamming.

How does Hamming Code Works?

- uses a block parity mechanism, data is divided into blocks, and parity is added to the block
- amount of parity data added to Hamming code is given by the formula $2^p \geq d + p + 1$
  - p is the number of parity bits
  - d is the number of data bits
- Hamming encoding with 4 data bits and 3 parity bits for a total of 7 total bits is given as Hamming(7,4)



HAMMING CODE STEPS

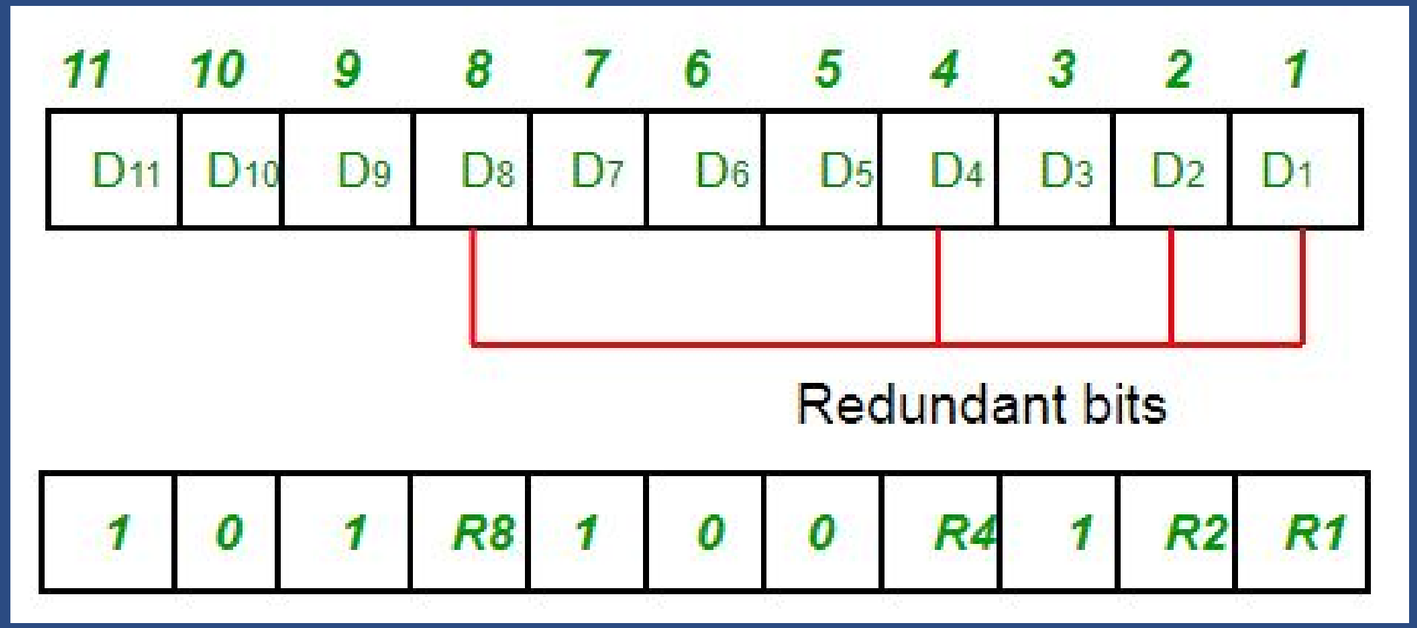
Given example: 1011001
(7 bits)

Hamming Code

STEP 1

Determine the number and position of redundant bits.

Redundancy bits are placed at positions that correspond to the power of 2.



The number of redundant bits is deduced from the expression $2^p \geq d + p + 1$:

$2^p \geq 7 + p + 1$

$2^p \geq 8 + p$

$2^4 \geq 8 + 4$

$16 \geq 12$

Therefore, the number of redundant bits is **4.**

Hamming Code

STEP 2

Determine the parity bit positions of redundant bits via table.

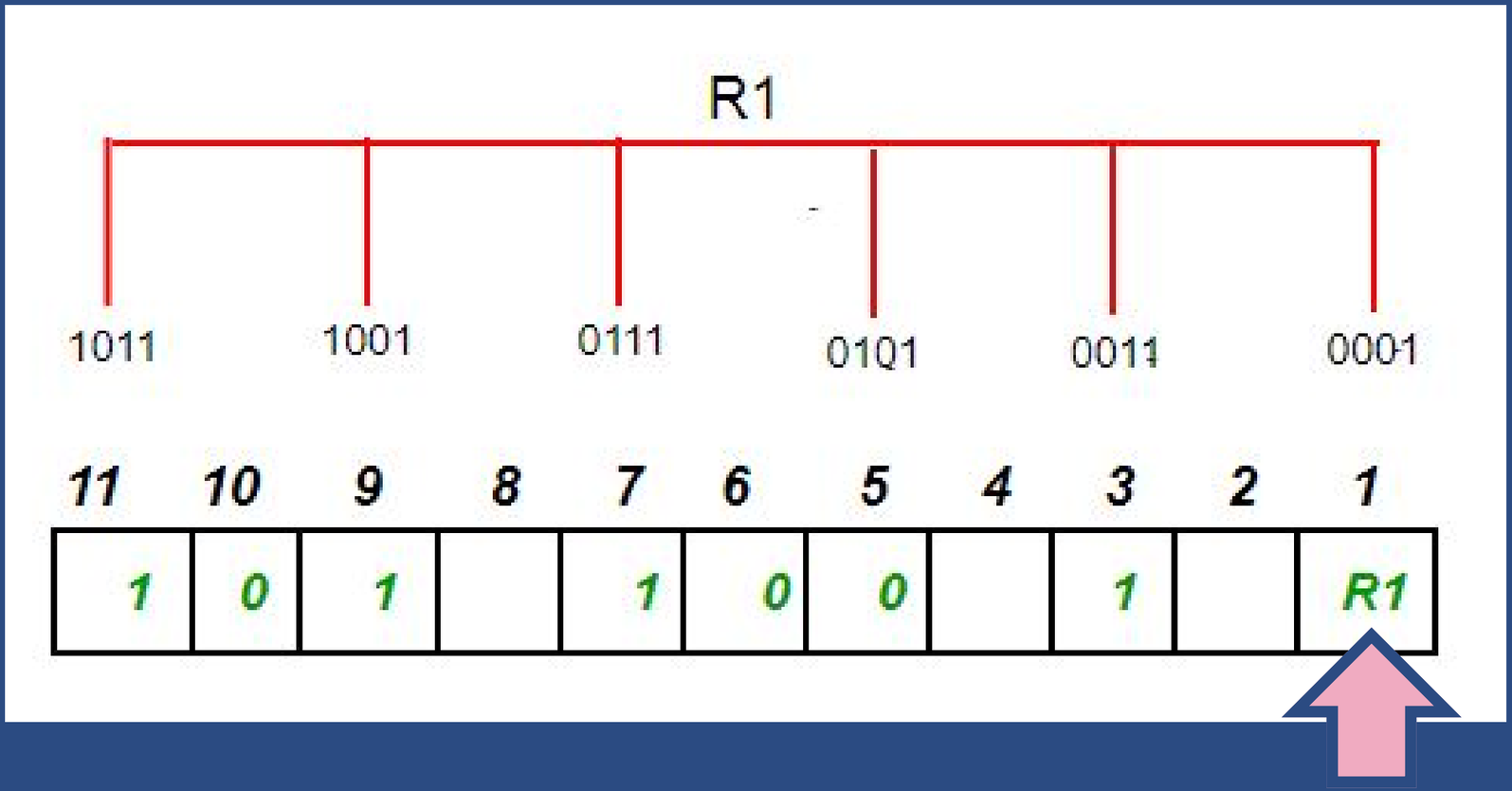
A parity bit is a bit appended to a data of binary bits to ensure that the total number of 1's in the data is even or odd.

R1 = 1,3,5,7,9,11
R2 = 2,3,6,7,10,11
R4 = 4,5,6,7
R8 = 8,9,10,11

Position	R8	R4	R2	R1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1

Hamming Code

STEP 3



Begin parity bit (Even Parity Bit)

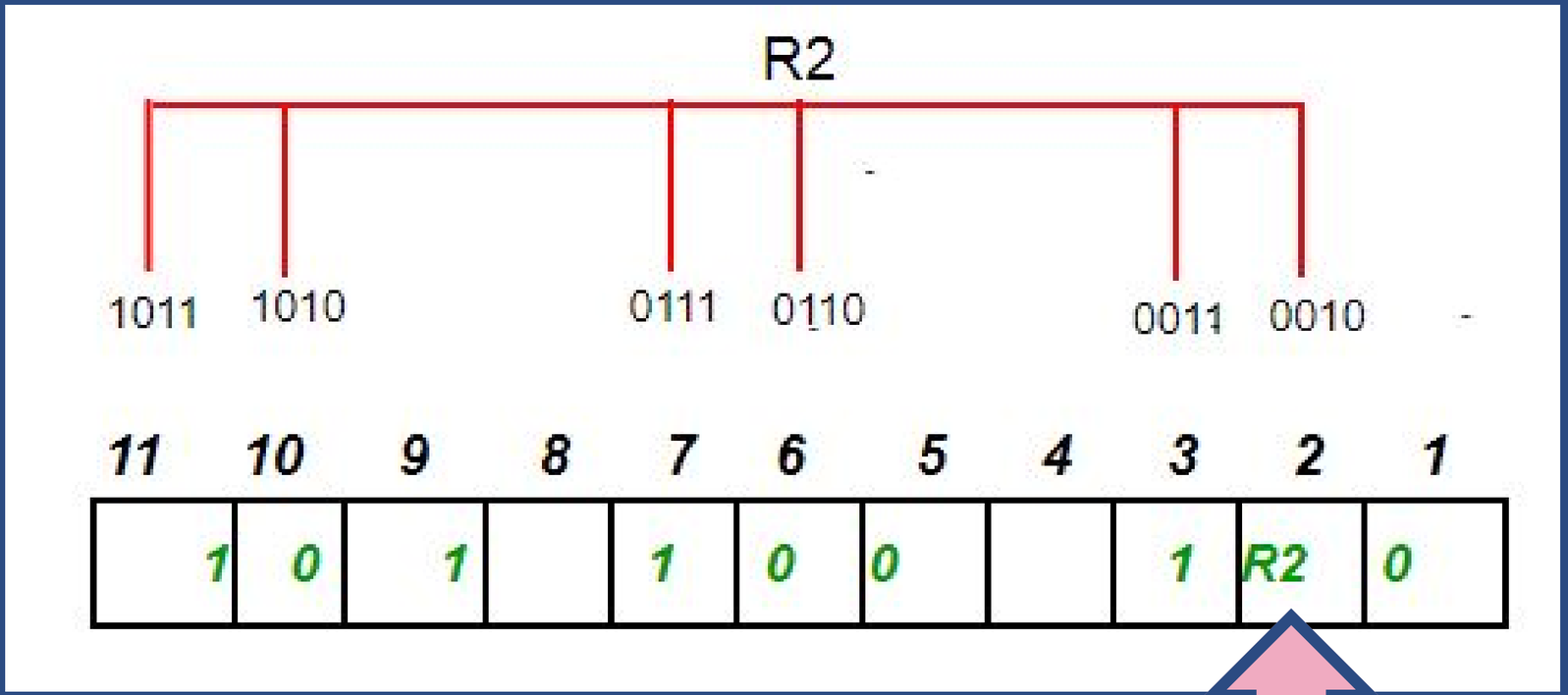
R1 = 1,3,5,7,9,11
R2 = 2,3,6,7,10,11
R4 = 4,5,6,7
R8 = 8,9,10,11

Total number of 1s: 4
Odd or Even: 4 is even
Therefore, value of R1: 0

Even Parity Rules
EVEN 1s = 0
ODD 1s = 1

Hamming Code

STEP 3



Begin parity bit (Even Parity Bit)

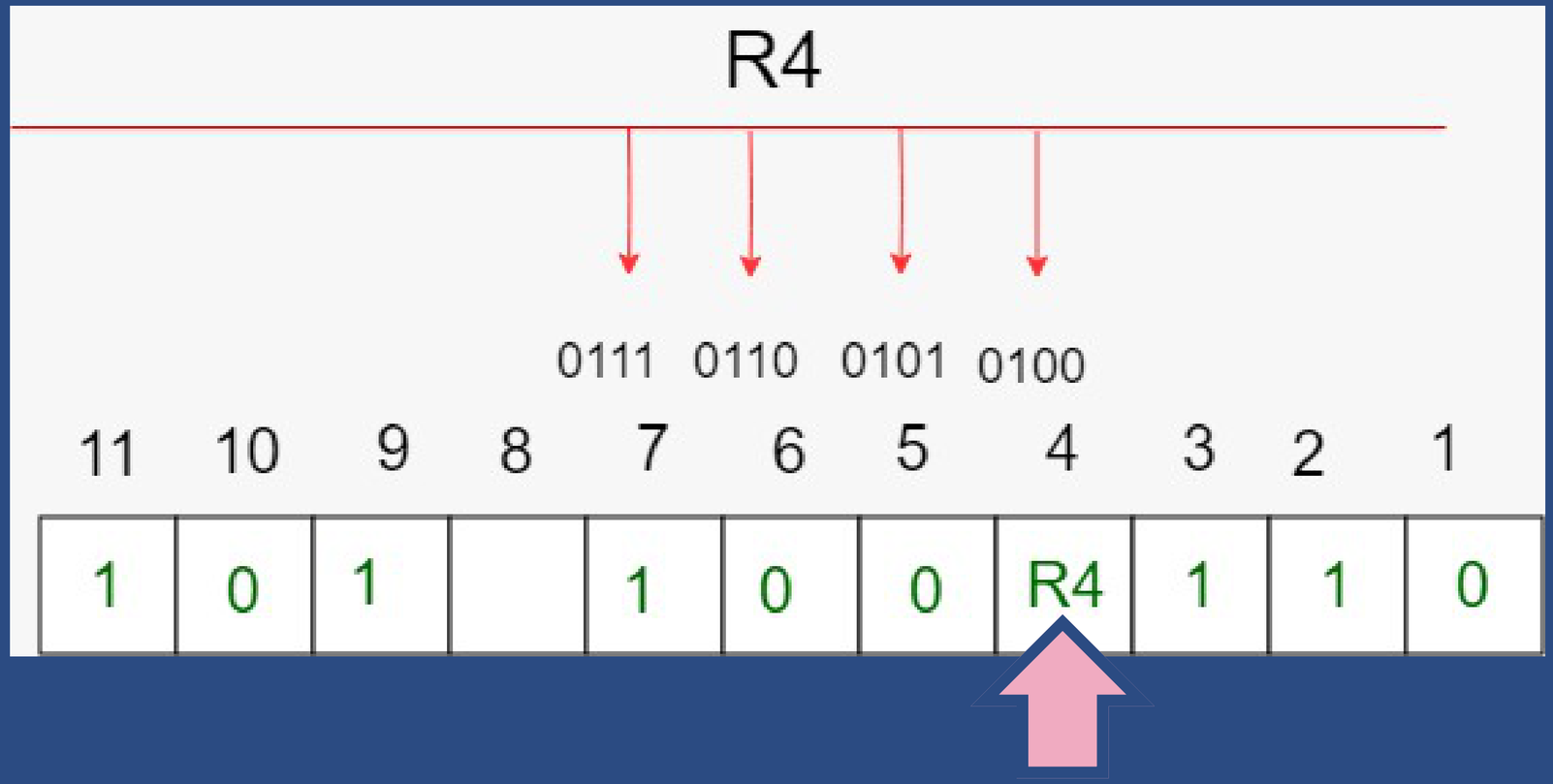
R1 = 1,3,5,7,9,11
R2 = 2,3,6,7,10,11
R4 = 4,5,6,7
R8 = 8,9,10,11

Total number of 1s: 3
Odd or Even: 3 is odd
Therefore, value of R2: 1

Even Parity Rules
EVEN 1s = 0
ODD 1s = 1

Hamming Code

STEP 3



Begin parity bit (Even Parity Bit)

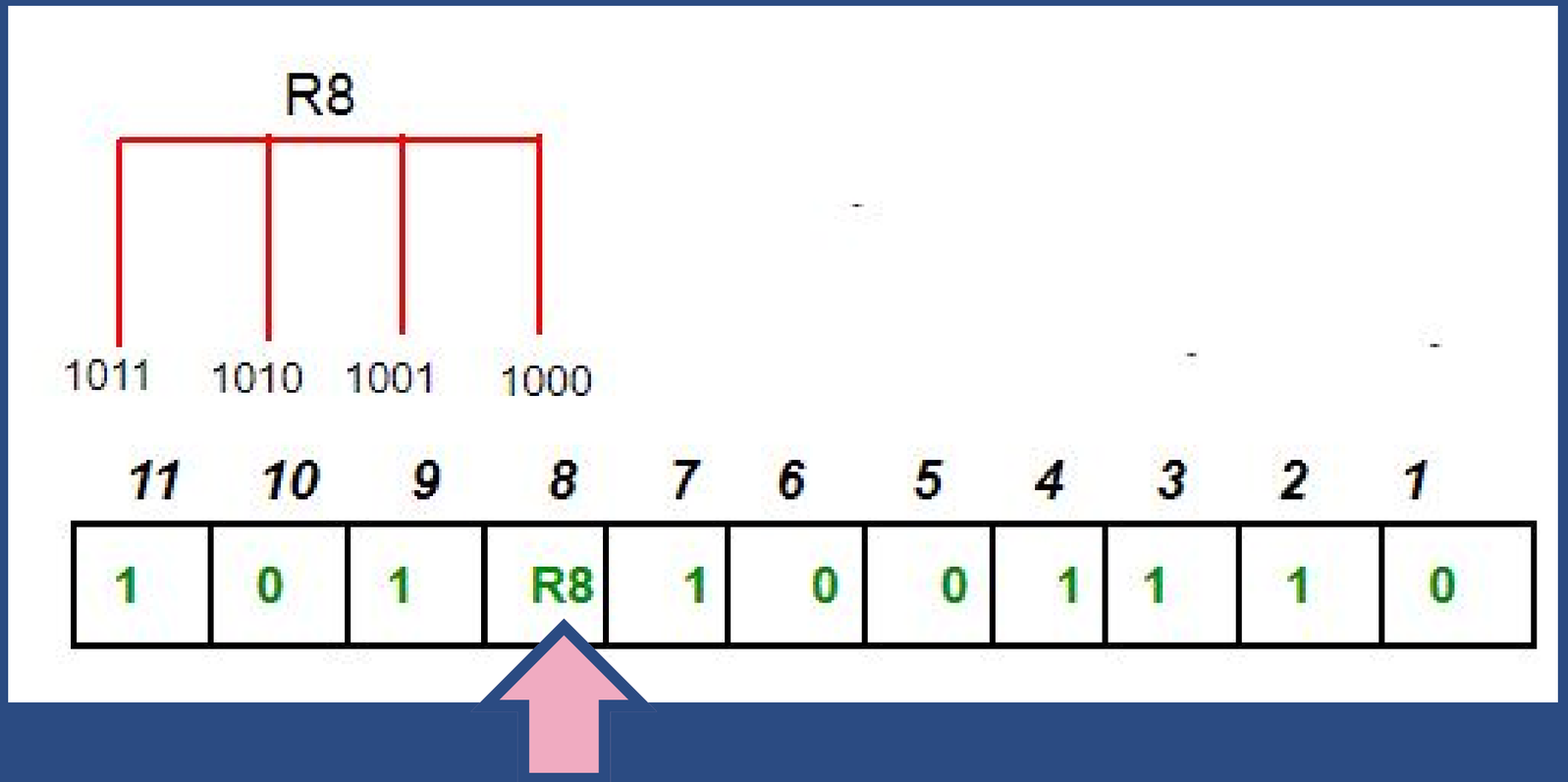
R1 = 1,3,5,7,9,11
R2 = 2,3,6,7,10,11
R4 = 4,5,6,7
R8 = 8,9,10,11

Total number of 1s: 1
Odd or Even: 1 is odd
Therefore, value of R4: 1

Even Parity Rules
EVEN 1s = 0
ODD 1s = 1

Hamming Code

STEP 3



Begin parity bit (Even Parity Bit)

R1 = 1,3,5,7,9,11
R2 = 2,3,6,7,10,11
R4 = 4,5,6,7
R8 = 8,9,10,11

Total number of 1s: 2
Odd or Even: 2 is even
Therefore, value of R8: 0

Even Parity Rules
EVEN 1s = 0
ODD 1s = 1

Hamming Code

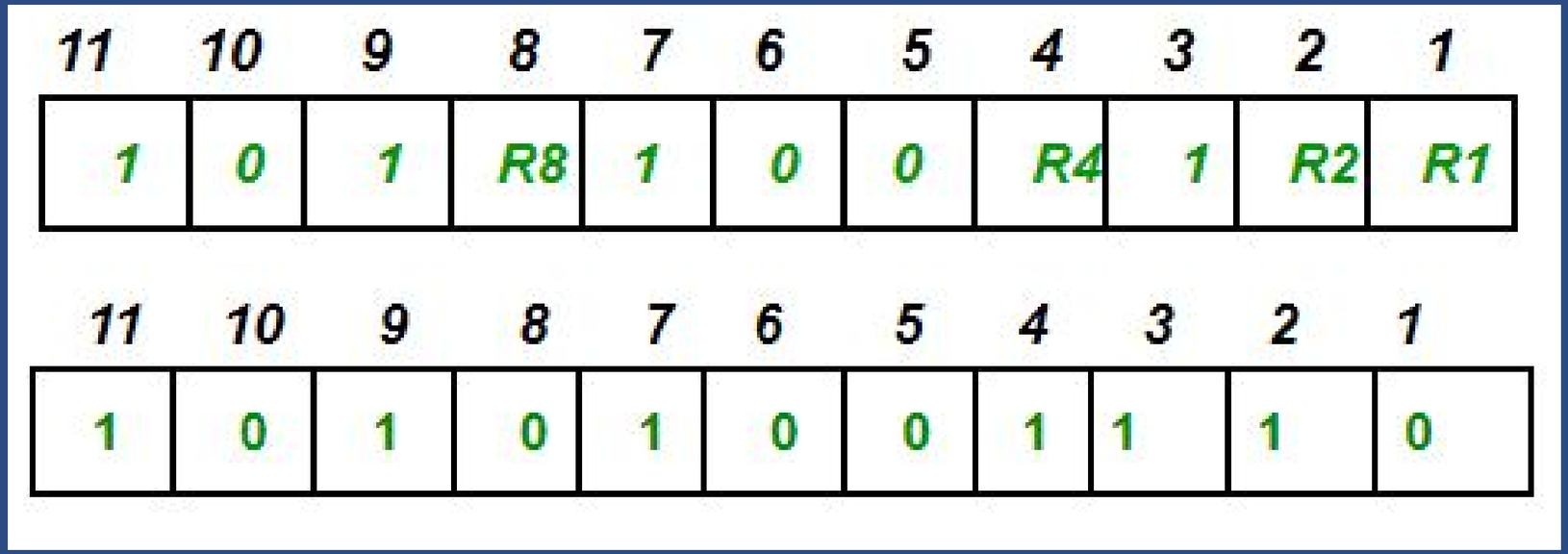
STEP 4

Finalize transferred data.

This is the finalized answer for the transferred data.

Final Values

R1 = 0
R2 = 1
R4 = 1
R8 = 0



THANK YOU

FOR LISTENING